

BDI Training System

Overview

The BDI Training System is a comprehensive framework for training native AI processes through structured, progressive learning. It consists of two main components:

1. **Training Tables:** Pre-generated datasets covering mathematics, logic, language, and code
2. **Curriculum Controller:** An 8-phase progressive learning system with adaptive advancement

Architecture

Training Tables

The system generates 12 binary training tables organized into 4 categories:

Math Tables (4 files)

- `math_arithmetic.dat` : Basic arithmetic operations (addition, subtraction, multiplication, division, exponentiation)
- `math_modular.dat` : Modular arithmetic operations
- `math_number_theory.dat` : GCD, LCM, primality testing, coprimality
- `math_sequences.dat` : Fibonacci, primes, squares, triangular numbers, factorials, Catalan numbers

Logic Tables (3 files)

- `logic_boolean.dat` : Two-valued boolean logic (AND, OR, NOT, XOR, etc.)
- `logic_three_valued.dat` : Three-valued Kleene logic with unknown values
- `logic_propositional.dat` : Tautologies, contradictions, logical equivalences

Language Tables (2 files)

- `language_ngrams.dat` : Character n-grams (unigrams through 4-grams) with frequency data
- `language_syntax.dat` : Grammatical patterns, case patterns, script transitions

Code Tables (3 files)

- `code_ast.dat` : Abstract syntax tree patterns for common code structures
- `code_idioms.dat` : Common programming idioms and patterns
- `code_bugs.dat` : Bug patterns for detection and prevention

Curriculum Controller

The curriculum controller manages AI process learning through 8 progressive phases:

Phase 0: Foundations

- Basic arithmetic (0-100)
- Simple boolean logic
- Character recognition
- **Gate:** 90% accuracy, 100 samples minimum

Phase 1: Elementary

- Extended arithmetic (0-1000)

- Boolean logic operations
- Bigrams
- **Gate:** 92% accuracy, 150 samples minimum

Phase 2: Intermediate

- Modular arithmetic
- Three-valued logic
- Trigrams
- **Gate:** 94% accuracy, 200 samples minimum

Phase 3: Advanced

- Number theory (GCD, LCM, primes)
- Propositional logic
- Syntax fragments
- **Gate:** 95% accuracy, 250 samples minimum

Phase 4: Expert

- Complex sequences
- Predicate logic patterns
- AST patterns
- **Gate:** 96% accuracy, 300 samples minimum

Phase 5: Master

- Advanced algorithms
- Complex logic
- Code idioms
- **Gate:** 97% accuracy, 350 samples minimum

Phase 6: Virtuoso

- Optimization patterns
- Proof patterns
- Bug detection
- **Gate:** 98% accuracy, 400 samples minimum

Phase 7: Transcendent

- Novel problem solving
- Creative synthesis
- Advanced reasoning
- **Gate:** 99% accuracy, 500 samples minimum

Data Format

All training tables use a consistent binary format:

File Header (64 bytes)

```
struct table_header {
    uint32_t magic;           // 0x42444954 ('BDIT')
    uint32_t version;         // Format version (1)
    uint32_t entry_type;      // Type of entries
    uint32_t num_entries;     // Number of entries
    uint64_t total_size;      // Total file size
    uint32_t checksum;        // CRC32 checksum
    uint32_t reserved[10];    // Reserved
};
```

Training Entry

```
struct training_entry {
    uint32_t entry_type;      // Type of training entry
    uint32_t difficulty;      // Difficulty level (0-7)
    uint32_t input_size;      // Size of input data
    uint32_t output_size;     // Size of output data
    uint8_t data[];           // Variable-length data
};
```

API Usage

Initializing the System

```
#include "curriculum.h"

// Initialize curriculum controller
curriculum_controller_t *ctrl = curriculum_init("/path/to/training/data");

// Register an AI process
curriculum_process_t *process = curriculum_register_process(ctrl, "ai_process_1");
```

Training Session

```
// Start a training session
curriculum_start_session(process);

// Get training examples
void *input, *output;
size_t input_size, output_size;
curriculum_get_next_example(process, &input, &input_size, &output, &output_size);

// Submit answer
bool correct;
curriculum_submit_answer(process, answer, answer_size, &correct);

// End session
curriculum_end_session(process);
```

Progress Tracking

```
// Get current phase
curriculum_phase_t phase = curriculum_get_current_phase(process);

// Get accuracy metrics
double phase_accuracy = curriculum_get_phase_accuracy(process, phase);
double topic_accuracy = curriculum_get_topic_accuracy(process, TOPIC_MATH);
double overall_accuracy = curriculum_get_overall_accuracy(process);

// Check if can advance
if (curriculum_can_advance_phase(process)) {
    curriculum_advance_phase(process);
}
```

Persistence

```
// Save progress
curriculum_save_progress(process, "progress.dat");

// Load progress
curriculum_load_progress(process, "progress.dat");
```

Building and Testing

Build the System

```
cd C/training
make all
```

Generate Training Data

```
make generate
```

This creates all 12 .dat files in `C/data/training/`.

Run Tests

```
make test
```

Validate Data

```
make validate
```

Performance Characteristics

- **Table Generation:** ~2-5 minutes for all 12 tables
- **Total Data Size:** ~300-500 MB
- **Curriculum Overhead:** <1% CPU usage
- **Progress Tracking:** Lock-free atomic operations where possible

- **Memory Usage:** ~10 MB per AI process

Integration with BDI Kernel

The training system integrates with the BDI kernel through:

1. **Memory Management:** Uses kernel allocators
2. **Process Management:** Registers as kernel processes
3. **Scheduling:** Cooperates with kernel scheduler
4. **Persistence:** Uses kernel filesystem interface

Future Enhancements

- Dynamic difficulty adjustment
- Multi-modal learning (combining topics)
- Transfer learning between phases
- Collaborative learning between processes
- Real-time performance analytics
- Adaptive curriculum generation

References

- BDI Architecture Documentation
- Unicode 17.0.0 Specification
- Training Table Format Specification
- Curriculum Design Document